

# Agentic research you can look back on

Work with your agent as naturally as with a colleague, and still get research you can rely on.

# What happens in this hour

- 01 Why: how research actually happens, and what agents change
- 02 r3, the core tool: it keeps the record of what was done
- 03 Helpers around r3
- 04 An agentic stack on top of r3
- 05 What you will do: install, models, tasks

# This is how research actually happens

```
project/  
  train.py  train_v2.py  train_final_FIXED.py  train_USE_THIS.py  
  eval.py   eval_old_dont_use.py  
  results/  results_FINAL/  results_FINAL_fixed/  
  model_best.pth  model_best_v2.pth  model_lr3e-4_bs64_actually_good.pth  
  Untitled7.ipynb  asdf.py  tmp2/  old/  backup/
```

Everything people complain about in vibe coding, we did by hand for decades.

No process, just people trying things. Nobody knows which train script made the paper's number.

# And the engineer's answer does not fit either

Testing a hunch, say dividing a score by a per-image constant, takes 30 seconds in a throwaway script and half a day in a mature library.

**quick-and-dirty hack**

**full software engineering**

right early, fatal if you stay

right for mature shared code, crippling for a quick pilot

**What we need: start dirty, tighten as the work matures, and never lose track of what was done.**

# Then the agent arrived

Far more results, faster, by a party that was not in the room.

- Maybe you still read what it writes. In a month, neither you nor the agent will know what it ran, on what, and why.
- Nobody re-reads a log of 400 tool calls.
- Close the session, and the reasoning and the loose ends go with it.

What was a nuisance becomes the bottleneck.

---

— When the work is done, can you still tell what was done?

# What would fix it: a record, kept for you

Not discipline. A record of what was actually done, written as a side effect of working.

**Provenance** is what went into each result: which data, which code, which earlier results it builds on. The chain runs all the way to the figure in the paper.

We built a tool for exactly this in our lab in 2022, before coding agents existed. It is called r3.

# The experiment is the unit

CLASSIC · REPO AS UNIT

```
project-repo/
  model.py
  train.py
  configs/
    sweep_02.yaml
  runs/
    lr3e-4/
    lr1e-4/
    eval_ckpt17/
```

OUTER The code

INNER The experiments



R3 · EXPERIMENT AS UNIT

```
job-13 · lower lr/ Frozen under its own id
  question.md
  deps/
    project-repo @ a3f9
    other-lab/eval @ v2.1
    baseline.ckpt from job-12
  run.sh Your script. r3 never runs it.
  output/ Its stored outputs stay here
```

OUTER What do I want to do?

INNER What do I need to do that?

job-12 → job-13 → job-14 · the report

— A project becomes a sequence of questions, each carrying exactly what it needed to answer it.

# r3 in four words

|                   |                                                                                                                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JOB</b>        | a self-contained directory: your code, the script that runs it, what it needs, declared. Each experiment, dataset or report is one job. Git repositories live inside it as pinned dependencies, not around it. |
| <b>COMMIT</b>     | freezes the job under an id: the code and what it declared. Then it runs once, and its outputs stay with it.                                                                                                   |
| <b>DEPENDENCY</b> | "the latest dataset under this name". Declared loosely, pinned to an exact id at commit.                                                                                                                       |
| <b>REPORT</b>     | a job whose output is a document, rendered from the stored outputs of the jobs it depends on.                                                                                                                  |

r3 never runs your job. Your script does. r3 keeps the record.

— Declare what you use. r3 remembers exactly which.

# The r3 tutorial, live

There is a tutorial: everything you need to know about r3, from a first job to scaling up.

- §1–2 · A download as a job: `r3 commit`, `bash run.sh`, `r3 find -l`
- §3.1 · You ask for "latest kodak"; the committed `r3.yaml` records which one
- §3.2 · `r3 checkout` : a working copy with the dependencies linked in
- §8 · The report is a job too: the page, then its `r3.yaml`
- Read later: §11 first (environment, cluster, library); §9, the refused delete

**LIVE** §1–2 the first job · §3 dependencies and checkout · §8 the report ↗

— Provenance does not stop at the compute job.

# A job is a directory; commit gives it an id

- `kodak/run.sh` is a download script. A job, because the script is the record of where the data came from.
- `metadata.yaml` gives it a name, `path: kodak`, that other jobs will use.
- `r3 commit kodak` copies it into the repository under an id. Then it runs there, once.

**LIVE** `kodak/run.sh · r3 commit · r3 find -l` ↗

# Declare loosely, freeze at commit

```
# you write
- find_latest: {path: kodak}
# r3 commits
- find_latest: {path: kodak}
  job: 82fd7a07-3190-445e-9081-f985cc2af74c
```

`r3.yaml` lists what a job needs. `find_latest` asks for the newest job with a name. `r3 checkout` builds a working copy with the dependencies linked in and `output/` pointing back into the store.

**LIVE** §3.1 the `r3.yaml` query · the committed `r3.yaml` ↗

**LIVE** §3.2 `r3 checkout` · `output/` linked back ↗

— You declare. r3 records.

# Provenance does not stop at the compute job

Most tools that track provenance stop at the trained model. `r3` keeps going.

- The analysis and the discussion are jobs too: a script that renders a report instead of training a model.
- The report depends on the result jobs, so every number in it traces back and cannot drift.
- `r3` has no opinion on what a job computes. So a Quarto report, or a notebook, can simply be one.

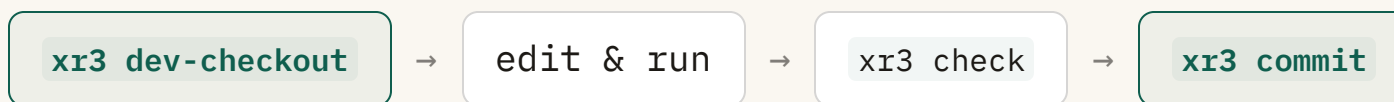
LIVE

the rendered page · its `r3.yaml`, where one `find_all` line collects both versions of the experiment (built in §6–7, which we skip) [↗](#)

— The write-up is part of the record, not a notebook on the side.

# A check before anything is frozen

r3 is not opinionated on purpose: it gives you the core and leaves the convenience layer to you. `xr3` is mine, and a sensible place to start.



## COMMAND

## WHAT IT DOES

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <code>find</code>   | query by tag, path pattern or JSON                      |
| <code>diff</code>   | this directory against its last commit                  |
| <code>check</code>  | version tag set, no buggy dependencies, git code pushed |
| <code>commit</code> | runs <code>check</code> , then commits                  |

— The check is the one that pays for itself.

LIVE · FOREMAN OVER MY REPOSITORY

# My repository, live

`foreman` : a web UI over the repository, with report links and dependency views. Not core `r3`, since you may prefer another interface.

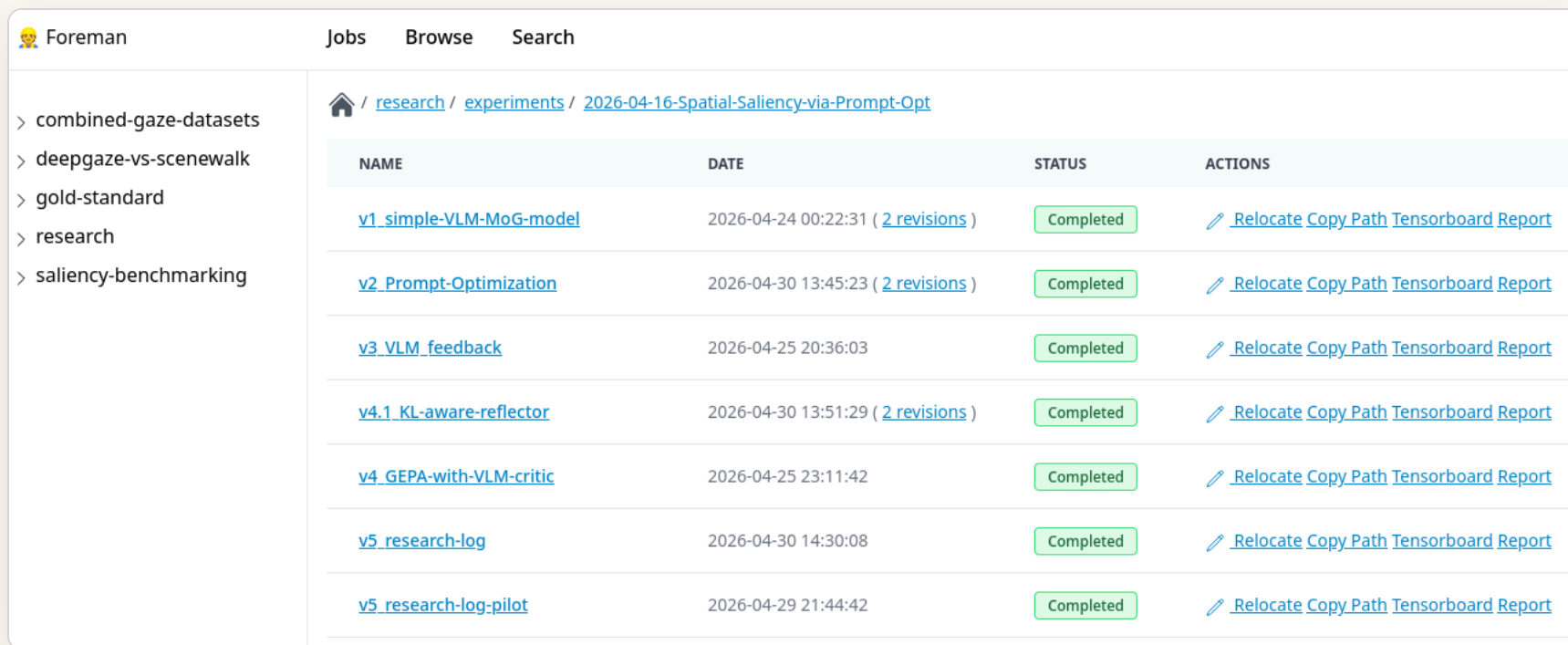
- 5398 committed jobs: the tutorial, in miniature
- `gold-standard` : a script grows across versions, then moves into the library
- `2026-01-09-refactor-to-library` : git code, datasets and the whole environment as dependencies
- A months-old failed run: its report still says what was tried, and what came next
- Dependencies, both ways: what one report rests on, 82 jobs; what a bug in a dataset reaches

LIVE `gold-standard` · the refactor job's `r3.yaml` · a failed run's report · Dependencies, both ways ↗

— A bug's reach is one query, not a week of archaeology.

# A web UI over your repository

`foreman` is a web UI over your repository. It is not part of core r3, since you may prefer a different interface. Every job has a direct link to its rendered report, and a dependency view, upstream and downstream, direct or recursive.



The screenshot shows the Foreman web interface. On the left is a sidebar with a user icon and the name 'Foreman', and a list of categories: 'combined-gaze-datasets', 'deepgaze-vs-scenewalk', 'gold-standard', 'research', and 'saliency-benchmarking'. The main area has tabs for 'Jobs', 'Browse', and 'Search'. Below the tabs is a breadcrumb path: [/](#) [research](#) / [experiments](#) / [2026-04-16-Spatial-Saliency-via-Prompt-Opt](#). A table follows with columns for NAME, DATE, STATUS, and ACTIONS. The table lists seven experiments, all with a 'Completed' status. Each row has a pencil icon followed by links for 'Relocate', 'Copy Path', 'Tensorboard', and 'Report'.

| NAME                                    | DATE                                                | STATUS    | ACTIONS                                                                                                                 |
|-----------------------------------------|-----------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#">v1_simple-VLM-MoG-model</a> | 2026-04-24 00:22:31 ( <a href="#">2 revisions</a> ) | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v2_Prompt-Optimization</a>  | 2026-04-30 13:45:23 ( <a href="#">2 revisions</a> ) | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v3_VLM_feedback</a>         | 2026-04-25 20:36:03                                 | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v4.1_KL-aware-reflector</a> | 2026-04-30 13:51:29 ( <a href="#">2 revisions</a> ) | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v4_GEPA-with-VLM-critic</a> | 2026-04-25 23:11:42                                 | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v5_research-log</a>         | 2026-04-30 14:30:08                                 | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |
| <a href="#">v5_research-log-pilot</a>   | 2026-04-29 21:44:42                                 | Completed | <a href="#">/</a> <a href="#">Relocate</a> <a href="#">Copy Path</a> <a href="#">Tensorboard</a> <a href="#">Report</a> |

One line of experiments, v1 to v5, browsed by path

IF THE LIVE VIEW FAILS

# The dependency view, both ways

Foreman

JobsBrowseSearch

Tensorboard

Report

Dependencies

Files

> output  
metadata.yaml  
r3.yaml  
report.qmd  
run.sh  
styles.css

Dependencies of gold-standard/experiments/2024-09-28\_performance\_gold\_standards\_with\_deepgaze

Depends on  
82 job(s) [direct](#) | [recursive](#)

10 entries per page

Search:

| UUID                                                 | Date                | Status    | Tags                                                                                                                                                                                                                                        | Actions                                                  |
|------------------------------------------------------|---------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <a href="#">c191f461-47ce-4499-99fb-770ceab2c167</a> | 2024-09-26 23:26:15 | Completed | gold-standard/experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_deepgaze_joint #experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_deepgaze_joint/v1.0.0 #gold-standard #mkuemmerer31                                    | <a href="#">Copy Path</a><br><a href="#">Tensorboard</a> |
| <a href="#">430c8597-0d31-45ad-8024-d09401de52a1</a> | 2024-09-26 23:25:53 | Completed | gold-standard/experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_joint #experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_joint/v1.0.0 #gold-standard #mkuemmerer31                                                      | <a href="#">Copy Path</a><br><a href="#">Tensorboard</a> |
| <a href="#">a0832601-6fa7-40ac-aa6f-ec0b58bff919</a> | 2024-09-23 00:14:20 | Completed | gold-standard/experiments/2024-07-31_gold_standard_MIT/uniform_centerbias #experiments/2024-07-31_gold_standard_MIT/uniform_centerbias/v1.0.0 #gold-standard #mkuemmerer31                                                                  | <a href="#">Copy Path</a><br><a href="#">Tensorboard</a> |
| <a href="#">357ebc69-0ff9-490d-ab17-b20b42d13605</a> | 2024-09-23 00:07:40 | Completed | gold-standard/experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_deepgaze #experiments/2024-07-31_gold_standard_MIT/uniform_centerbias_deepgaze/v1.0.0 #gold-standard #mkuemmerer31<br>combined-gaze-datasets/experiments/2024-07- | <a href="#">Copy Path</a><br><a href="#">Tensorboard</a> |

What one report rests on: 82 jobs, recursive

Foreman

JobsBrowseSearch

Tensorboard

Dependencies

Files

> output  
metadata.yaml  
r3.yaml  
run.py  
run.sh  
run\_inner.sh

Dependencies of combined-gaze-datasets/task0001\_datasets/CAT2000

Depended on by  
22 job(s) [direct](#) | [recursive](#)

10 entries per page

Search:

| UUID                                                  | Date                | Status    | Tags                                                                                                                                                                                                                                                   |
|-------------------------------------------------------|---------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">a0211063-2ceb-40ac-88a9-a2f8fd9f6da</a>   | 2024-01-14 18:31:01 | Completed | combined-gaze-datasets/task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_D #task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_DAEMONS_FI combined-datasets #original_CAT2000_dataset_experiment #hkhanuja78 #Imdb        |
| <a href="#">72bfddd0-8642-4e4e-b071-cabfa9f60f8c</a>  | 2024-01-10 11:22:41 | Completed | combined-gaze-datasets/task0001_datasets/CAT2000 #task0001_datasets/CAT2000_cleaned_v1 #cleaning_step_1 #removing_artefacts #dataset                                                                                                                   |
| <a href="#">bcd8b45c-3de9-47f1-9d63-ad9bcb3c30074</a> | 2023-12-21 20:53:03 | Completed | combined-gaze-datasets/task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_D #task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_DAEMONS_FI combined-datasets #original_CAT2000_dataset_experiment #hkhanuja78 #Imdb        |
| <a href="#">35841ff3-b80a-403e-a593-28c8caf7a477</a>  | 2023-12-18 10:15:31 | Completed | combined-gaze-datasets/task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_D #task0001_datasets/Combined_dataset_MIT1003_CAT2000_COCO_Freeview_DAEMONS_FI combined-datasets #original_CAT2000_dataset_experiment #hkhanuja78 #bug/1_DAEMO |
| <a href="#">93e5467c-3625-42d9-ad28-</a>              | 2023-12-07          | Failed    | combined-gaze-datasets/task0001_datasets/Combined_dataset #task0001_datasets/Combined_dataset #gaze-combined-datasets #original_CAT2000 da                                                                                                             |

What was built on one dataset: the jobs a bug in it would reach

— What a bug reaches is one view, not a week of archaeology.

# r3 was built before agents. It was waiting for them.

**PREVIEW****the observer**

learns how you work from your sessions; you approve each change

**EDITABLE****the how-we-work file**

our habits: jobs, reports, what counts as evidence

**FIXED****the r3 skill**

teaches any agent the four words

**THE HELPERS****xr3 · foreman**

what you just saw

**UNCHANGED****r3**

your experiments, frozen and findable

**TODAY**

r3 was built in 2022 for the mess from the directory slide. An agent makes the same mess, faster, while you are not watching. r3 fits as it is. Two files on top teach the agent to use it. The top layer is a preview.

# Two files teach the agent r3 and our habits

## THE R3 SKILL

- Read by the agent whenever r3 comes up
- Job, commit, dependency, report, plus queries and gotchas
- Pure r3: examples and suggestions, no rules

## RESEARCH\_WORKFLOW.MD

Meant to be edited. It gives the agent:

- a report template: tl;dr, then method, results, discussion, follow-up ideas
- numbers computed, never pasted; re-render without rerunning the compute
- a default workflow for experiments: spec to commit
- recipes for common job kinds: data, environment, compute, report
- a workflow for promoting throwaway code into a library

Two rules underneath: real work lives in jobs; every number traces to a job.

# Fast and intuitive, and still trustworthy

Work with your agent as naturally as with a colleague, while the setup shapes its work into research you can rely on.

- "Start a toy research project: evaluate a bunch of old-school natural image statistics models on natural images..."
- "could we think of a really good topline model ... maybe a normalizing flow?"
- "document the failure of a somewhat naive flow as an experiment..."

I wrote no code myself. I installed the toolchain, talked, and read reports.

## WHAT IT LEFT BEHIND

### A normalizing flow as a topline — it doesn't get there

It overfits, needs lots of data just to match ICA, and saturates far short of the radial model

PUBLISHED  
September 30, 2026

#### TL;DR

A naive attempt at a strong **topline** density model — a **normalizing flow** — **fails here**. It **overfits** (train NLL keeps dropping while held-out rises), and although with enough data it matches the *linear* models, it **saturates ~0.3 bits/component above SSD** — it never approaches the 2-parameter radial model. A flexible deep model losing to a 2-parameter model is the real result; a useful topline must build the radial/contrast structure in.

One of the reports: a candidate model that did not work, documented.

39 committed jobs, every number traceable to the job that computed it.

— Looking back, I can still tell what the agent did, and where every number came from.

# The stack should learn how you work

## MY PROOF OF CONCEPT

- an observer skill logs friction while I work
- a shared folder collects the logs; I review them and approve changes
- one person's practice, not shareable

## A WORKSHOP VERSION FOR YOU

- opt-in observer skill in our tooling
- logs friction with r3, the skill, the agent
- to a local file; you choose what to share

Not in the toolchain yet; the camp repo will announce it.

# One command installs the toolchain

```
curl -fsSL https://raw.githubusercontent.com/matthias-k/r3-tooling/builder-camp/bootstrap.sh | bash
```

Enter accepts each default · `--dry-run` to preview · `--yes` for all defaults

- `r3`, `xr3`, `foreman` on your PATH, a config file, a fresh repository
- the `r3` skill, so Claude Code or Codex knows how to use `r3`
- our research workflow, and one line in `CLAUDE.md` so every session follows it

Models: `https://llm.mlcloud.uni-tuebingen.de`, OpenAI-compatible, three open-weight models; I can provide keys.

---

— One command: the tools, the agent skill, and our workflow.

# Bring your own project, or start from ours

[github.com/matthias-k/AI-Builder-Camp-2026](https://github.com/matthias-k/AI-Builder-Camp-2026)

- 01 **Natural image statistics:** classic image models, on a laptop
- 02 **How much is a prompt worth:** format sensitivity on the hosted models
- 03 **How random is an LLM:** can it give you the distribution you ask for

Plus ideas for going further and a list of datasets. Whatever you do, keep it in r3.

# You are the first cohort: tell us how it went

We want to hear:

- what you liked about working with r3 and your agent
- where r3 got in your way
- where the agent went wrong, and whether the record caught it
- what you wish the how-we-work file had told your agent

Tell us through the workshop observer (opt-in; it logs to a local file you choose to share), or grab one of us in the work sessions.

# Everything rests on a record of what was actually done



[github.com/matthias-k/AI-Builder-Camp-2026](https://github.com/matthias-k/AI-Builder-Camp-2026)

So that next month you can still tell what was done, including what your agent did today.

Everything for this week is in one place, the camp repository:  
[github.com/matthias-k/AI-Builder-Camp-2026](https://github.com/matthias-k/AI-Builder-Camp-2026)

The tutorial and the toolchain are linked from there.



IF SOMEONE ASKS

# Why not just git and wandb?

They record everything  
except the thing you need.

| TOOL              | WHAT IT RECORDS                                               | WHAT IT CANNOT TELL YOU                                                           |
|-------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| git               | Versions of code. One working tree at a time.                 | Which commit of which repo made this checkpoint, on which data version.           |
| wandb             | Metrics of a run, linked to one commit hash.                  | The inputs behind the hash: other repos, data, environment.                       |
| runs/ · HF<br>hub | The outputs, named by hand:<br><code>model_best_v2.pth</code> | Which <code>train_*.py</code> made it. Rename a path, and every reference breaks. |

The reason is structural: git makes the repo the unit and experiments live inside it. r3 makes the experiment the unit, and inside it sits exactly what it needed: any repo at any version, data, a checkpoint from the job before.

— Which exact code, data and environment made this result? Today the answer is grep and memory.